

The Citadel System



Bruno Xampeny

Alex Ortiz

Sistemes Operatius – Curs 25/26

Índex

1. Disseny de la pràctica	1
1.1 Visió general i arquitectura	1
1.2 Processos i comunicació entre processos	2
1.2.1 Procés principal	2
1.2.2 Thread servidor	2
1.2.3 Processos Envoy	3
1.3 Protocol de comunicació	4
1.4 Estructures de dades i justificació	5
1.4.1 Route	6
1.4.2 Kingdom	6
1.4.3 Product	7
1.4.4 TradeList	8
1.4.5 AllianceStatus i AllianceRecord	8
1.4.6 EnvoyTask i EnvoyResult	9
1.4.7 Envoy	10
1.4.8 GlobalContext	10
1.5 Recursos del sistema utilitzats i justificació	11
1.5.1 Sockets	11
1.5.2 Threads	12
1.5.3 Forks	12
1.5.4 Pipes	12
1.5.5 Select	12
1.5.6 Senyals	12
1.5.7 Semàfors	13
1.5.8 System calls i utilitats	13
1.6 Fluxos principals	13
2. Problemes observats i com s'han solucionat	14
2.1 Bloqueig del programa per operacions de xarxa	14
2.2 Problemes de concurrència en la sortida per pantalla	14
2.3 Gestió complexa de comunicació amb els envois	15
2.4 Validació del protocol i corrupció de dades	15
2.5 Gestió de memòria dinàmica i fuites	15
2.6 Control del flux del mode trade	16
2.7 Gestió de senyals	16

2.8 Dificultat en el disseny global del sistema	16
3. Estimació temporal del desenvolupament.....	16
4. Conclusions i propostes de millora.....	17
5. Explicació dels noms dels processos i temàtica de la pràctica.....	18
6. Bibliografia utilitzada	19

1. Disseny de la pràctica

1.1 Distribució general i arquitectura

La pràctica s'ha dissenyat com un programa principal anomenat Maester, que representa un regne.

El Maester és el procés principal del sistema i s'encarrega de gestionar el funcionament general del regne. Entre les seves funcions principals hi ha la càrrega de la configuració del regne i de les seves rutes a partir d'un fitxer .dat, així com la càrrega dels productes locals des d'un fitxer.db.

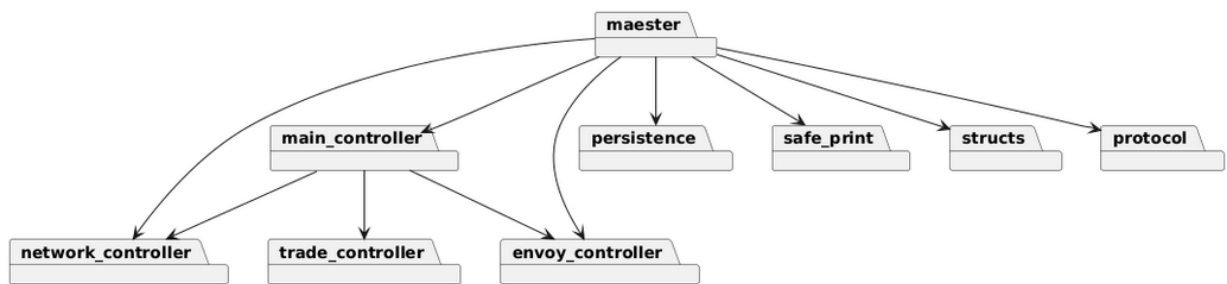
A més, el Maester permet a l'usuari interactuar amb el sistema mitjançant comandes introduïdes per terminal, com ara list, pledge o start trade. També gestiona la comunicació amb altres regnes a través de sockets TCP, tant per enviar com per rebre missatges.

Per evitar bloquejos en el flux principal, les tasques que poden ser llargues o bloquejants es deleguen als envoys, mentre que les connexions entrants es gestionen mitjançant un servidor que s'executa en un thread separat. Finalment, el Maester assegura una sortida per pantalla clara i ordenada mitjançant un sistema d'impressió sincronitzada.

A nivell de fitxers, el projecte queda organitzat així:

- **maester.c**: punt d'entrada del programa. Inicialitza context, carrega dades, crea el thread servidor i engega el menú.
- **main_controller.c/.h**: gestió del menú principal i parseig de comandes de l'usuari.
- **network_controller.c/.h**: comunicació per xarxa (server TCP, accept, send, i rutes).
- **protocol.c/.h**: definició de trames, serialització/deserialització i checksum.
- **envoy_controller.c/.h**: gestió del pool d'envoys (fork + pipes), assignació de tasques i recepció de resultats.
- **trade_controller.c/.h**: mode trade, gestió de llista local i verificació d'stock remot a través del catàleg.
- **persistence.c/.h**: lectura de Kingdom i Product, i guardat de fitxers temporals / persistència.
- **safe_print.c/.h**: imprimeix de forma thread-safe amb semàfor.
- **auxiliars.c/.h**: funcions utilitàries (read_until, to_lowercase, get_md5, etc.).
- **structs.h**: definició de totes les estructures i constants de suport.

Figure 1. Arquitectura global i dependències entre mòduls del sistema



1.2 Processos i comunicació entre processos

El disseny del sistema fa un ús explícit de la concurrència per garantir que el flux principal del programa no quedi bloquejat durant l'execució. Aquesta concurrència es gestiona a dos nivells diferenciats: d'una banda, mitjançant un thread servidor que s'encarrega de la comunicació de xarxa, i de l'altra, mitjançant processos Envoy independents que executen tasques potencialment llargues o bloquejants.

1.2.1 Procés principal

El procés principal és l'encarregat d'inicialitzar i coordinar el funcionament general del sistema. En primer lloc, inicialitza el GlobalContext i carrega la informació del regne i dels productes disponibles a partir dels fitxers corresponents.

Un cop inicialitzat l'estat del sistema, el procés principal engega el servidor mitjançant la creació d'un thread amb `pthread_create`, permetent que les connexions entrants es gestionin de manera concurrent. A continuació, entra al menú principal, des d'on es llegeixen i es processen les comandes introduïdes per l'usuari.

Quan el programa finalitza, el procés principal s'encarrega de tancar correctament tots els recursos oberts i d'alliberar la memòria utilitzada. Per aquest motiu, aquest procés actua com el "cervell" del sistema i és el principal punt d'interacció amb l'usuari.

1.2.2 Thread servidor

El servidor s'executa en paral·lel al procés principal dins d'un thread separat. El seu objectiu principal és gestionar la comunicació de xarxa amb altres regnes sense bloquejar el flux principal del programa.

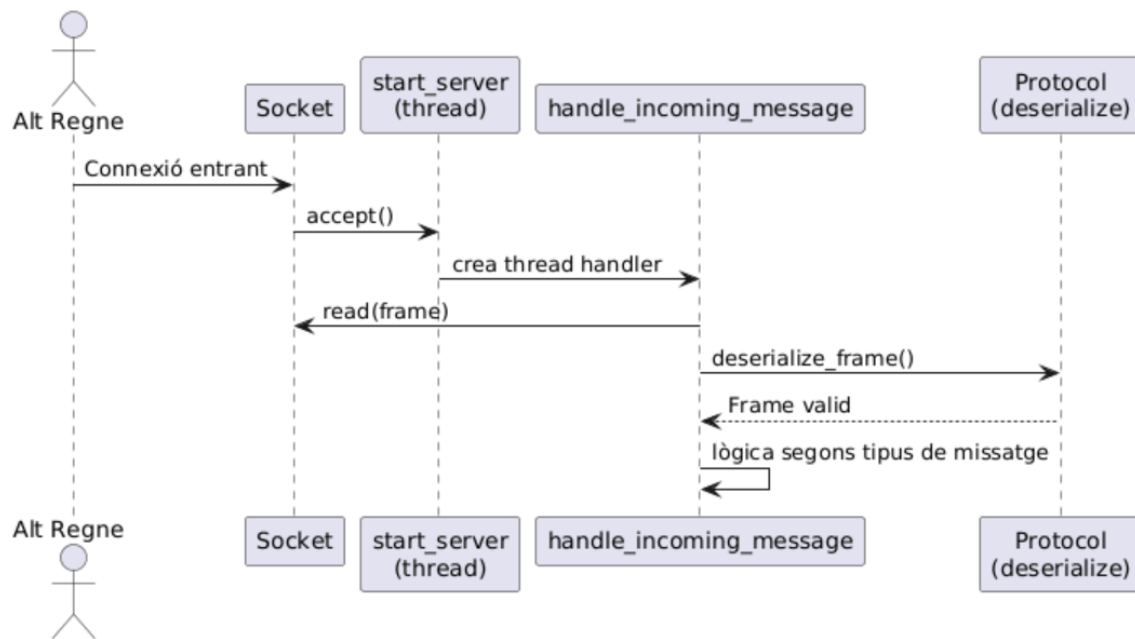
Aquest thread obre un socket d'escolta utilitzant les crides `bind()` i `listen()`, i accepta les connexions entrants d'altres regnes. Un cop establerta una connexió, el servidor llegeix els frames rebuts i determina el tipus de missatge, com ara peticions de pledge, llistats de productes o operacions de trade.

En funció del missatge rebut, el servidor actualitza l'estat intern del sistema, gestionant aliances, catàlegs o confirmacions. Quan és necessari, també pot

notificar els envois o imprimir informació per pantalla, permetent una gestió eficient i concurrent de les comunicacions de xarxa.

Això evita que el menú principal quedi bloquejat esperant connexions de xarxa: encara que el Maester estigui esperant comandes, el servidor pot rebre i processar missatges.

Figure 2. Diagrama de seqüència del tractament d'una connexió entrant al servidor



1.2.3 Processos Envoy

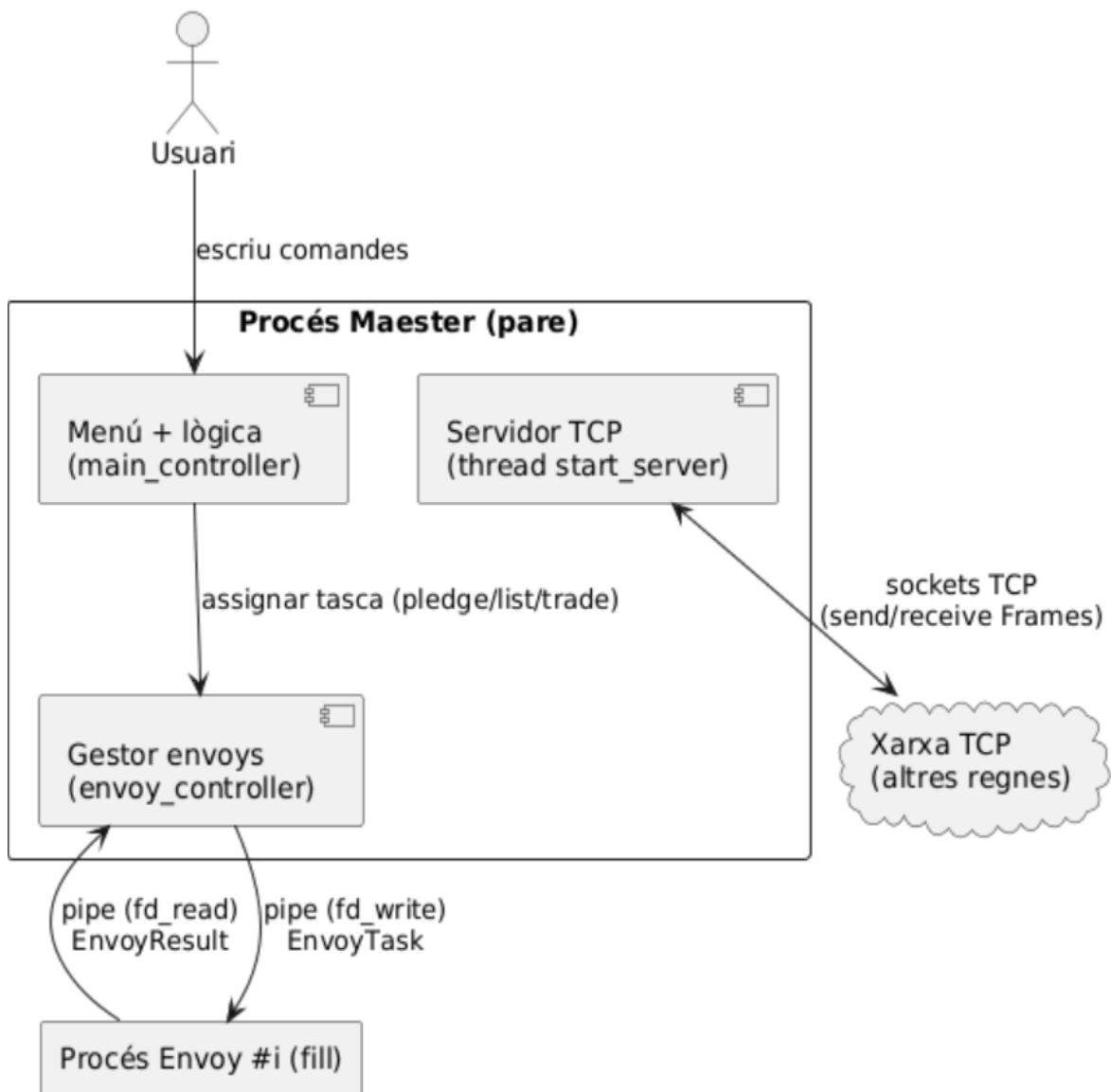
Els envois estan pensats per gestionar tasques que poden requerir temps o períodes d'espera, com ara enviar una petició de pledge i esperar una resposta amb timeout, demanar el catàleg de productes a un altre regne o enviar una llista de trade i esperar la seva confirmació.

Quan s'inicialitzen els envois, es crea un pool de processos. Per a cada envoy es defineixen dos canals de comunicació mitjançant pipes: un descriptor utilitzat pel Maester per enviar tasques a l'envoy i un altre descriptor perquè l'envoy pugui retornar els resultats un cop finalitzada la tasca.

Quan un envoy rep una tasca, aquest s'encarrega de processar-la de manera independent, establint la comunicació de xarxa necessària i enviant els frames o fitxers corresponents. En cas necessari, l'envoy pot esperar una resposta utilitzant `select()` amb un temps límit definit. Un cop la tasca acaba, retorna un `EnvoyResult` amb informació sobre si l'operació ha tingut èxit, el missatge associat i el regne implicat.

Aquest disseny permet que el menú principal no quedi bloquejat, possibilita tenir diversos envois executant tasques en paral·lel i assegura que cada envoy es mantingui ocupat fins que finalitza completament la tasca assignada.

Figure 3. Arquitectura de processos i comunicació del sistema



1.3 Protocol de comunicació

La comunicació entre regnes es realitza mitjançant un protocol propi basat en frames de mida fixa de 320 bytes, definit al fitxer protocol.h. Cada frame està format pels següents camps:

- type (1 byte)
- origin (20 bytes)
- destination (20 bytes)
- data_length (2 bytes)
- data (275 bytes)

- checksum (2 bytes)

Tal com es mostra a la Figure 4, aquest disseny permet que tots els missatges tinguin una mida constant, fet que simplifica la lectura per sockets i evita problemes associats a lectures parcials. A més, l'ús d'offsets definits garanteix una serialització consistent, mentre que el camp checksum permet detectar possibles corrupcions de dades durant la transmissió.

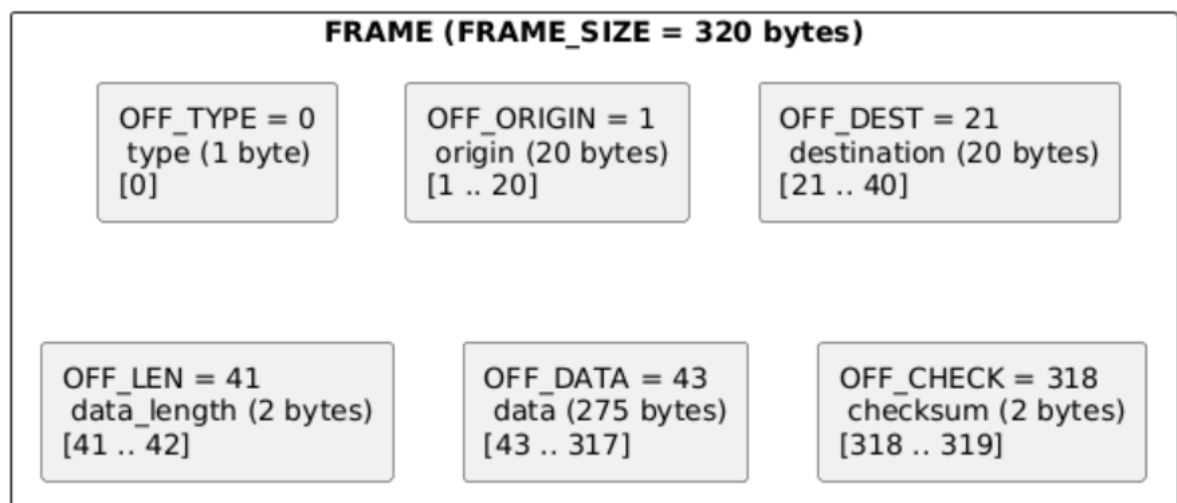
El procés de serialització consisteix a omplir el buffer del frame utilitzant offsets constants. Primer s'escriuen els camps type, origin, destination i data_length, a continuació es copia el contingut del camp data i finalment es calcula el checksum com la suma dels bytes del frame (mod 65536), que s'afegeix al final del buffer.

Durant la deserialització, el sistema valida el checksum del frame rebut abans de processar-lo. Si la validació és correcta, s'extreuen els diferents camps del buffer i el missatge es considera vàlid; en cas contrari, el frame es descarta.

Tipus principals que s'utilitzen:

- TYPE_PLEDGE_REQ (0x01) per demanar aliança
- TYPE_RESPONSE (0x03) per acceptar/rebutjar
- TYPE_LIST_PRODUCTS_REQ (0x11) per demanar catàleg
- TYPE_LIST_PRODUCTS_RESP (0x12) per respondre amb catàleg
- TYPE_SHOPPING_HEADER/RESPONSE per trade
- TYPE_DISCONNECT per avisar desconnexió

Figure 4. Estructura del frame del protocol de comunicació



1.4 Estructures de dades i justificació

A structs.h hi ha les estructures principals. Les he entès i les resumeixo amb el seu ús real dins del flux del programa:

1.4.1 Route

Figure 5. Definició de l'estructura Route

```
/* *****  
 *  
 * @Finalitat: Representa una ruta de comunicació entre regnes,  
 *             incloent el destí, la seva IP i el port de connexió.  
 *  
 * *****/  
  
typedef struct {  
    char *dest;    //Nom del regne de destí.  
    char *ip;      //Adreça IP del regne de destí.  
    int port;      //Port utilitzat per la comunicació.  
} Route;
```

L'estructura Route s'utilitza per representar una ruta cap a un regne destí. Conté la informació bàsica necessària per poder establir una comunicació amb un altre regne, com ara el seu nom, l'adreça IP i el port de connexió.

Aquesta estructura és necessària perquè el Maester pugui saber a quin regne s'ha de connectar quan vol enviar un frame. Dins del sistema, s'utilitza principalment per localitzar la ruta correcta mitjançant la funció `find_route_for` del mòdul `network_controller`, així com per construir la taula inicial d'aliances entre regnes.

Separar la informació de les rutes de la resta de dades del regne permet gestionar fàcilment múltiples destins i manté el disseny del sistema senzill i clar.

1.4.2 Kingdom

Figure 6. Definició de l'estructura Kingdom

```
/* *****  
 *  
 * @Finalitat: Conté tota la informació d'un regne,  
 *             com el nom, camí de fitxers, enviats,  
 *             adreça IP, port i les seves rutes disponibles.  
 *  
 * *****/  
  
typedef struct {  
    char *name;    //Nom del regne.  
    char *path;    //Ruta d'emmagatzematge associada al regne.  
    int envoys;    //Nombre d'enviats disponibles.  
    char *ip;      //Adreça IP del regne.  
    int port;      //Port de comunicació del regne.  
    Route *routes; //Llista de rutes de comunicació amb altres regnes.  
    int n_routes;  //Nombre total de rutes.  
} Kingdom;
```

L'estructura Kingdom defineix la configuració del regne local dins del sistema. En ella s'emmagatzema tota la informació necessària per al funcionament del regne, com ara el seu nom, les rutes de fitxers associades, el nombre d'envois disponibles, així com l'adreça IP, el port del servidor i la llista de rutes cap a altres regnes.

Aquesta estructura permet centralitzar tota la configuració del regne en un únic punt i facilita la càrrega de dades a partir del fitxer .dat durant la inicialització del programa. Dins del projecte, Kingdom s'utilitza de manera transversal en diversos mòduls, com el maester, el main_controller, el network_controller o l'envoy_controller, ja que moltes operacions depenen directament de la configuració del regne.

Considerar Kingdom com una unitat pròpia permet identificar clarament cada regne i simplifica la realització de proves amb diferents configuracions mitjançant diversos fitxers .dat.

1.4.3 Product

Figure 7. Definició de l'estructura Product

```

/*****
 *
 * @Finalitat: Representa un producte dins del sistema de comerç,
 *             incloent el seu nom, quantitat i pes.
 *
 *****/

typedef struct {
    char name[100]; //Nom del producte.
    int amount;      //Quantitat disponible del producte.
    float weight;     //Pes del producte.
} Product;

```

L'estructura Product representa un producte local del regne i emmagatzema la informació bàsica associada a cada element, com el seu nom, la quantitat disponible i el pes.

Aquesta estructura s'utilitza per mostrar els productes locals disponibles i per gestionar les operacions de comerç durant el mode trade. Dins del sistema, Product es fa servir tant en el llistat de productes com en la persistència de dades al fitxer .db, així com en les operacions relacionades amb el trade.

El fet que sigui una estructura compacta permet llegir i escriure directament els productes de manera binària, utilitzant operacions com `read()`, simplificant la persistència i millorant l'eficiència del sistema.

1.4.4 TradeList

Figure 8. Definició de l'estructura TradeList

```
/* *****  
 *  
 * @Finalitat: Defineix un element d'intercanvi dins de la llista de comerç,  
 *             indicant quin producte s'intercanvia i en quina quantitat.  
 *  
 ***** */  
  
typedef struct {  
    Product item;    //Producte associat a l'intercanvi.  
    int amount;      //Quantitat del producte per intercanviar.  
} TradeList;
```

L'estructura TradeList representa un element de la llista de trade, indicant quin producte es vol intercanviar i en quina quantitat. Aquesta llista es construeix de manera progressiva quan l'usuari es troba en mode trade i va afegint els diferents ítems que vol enviar a un altre regne.

Dins del sistema, la llista de trade s'utilitza com una estructura temporal gestionada pel trade_controller, a través de les variables ctx->trade_list i ctx->trade_count, i també es fa servir en la generació del fitxer temporal que s'envia durant l'operació de trade.

Separar la llista de trade de l'inventari local permet mantenir una gestió més clara del comerç, evitant modificar els productes del regne fins que el trade es completa correctament.

1.4.5 AllianceStatus i AllianceRecord

Figure 9. Definició de AllianceStatus i AllianceRecord

```
/* *****  
 *  
 * @Finalitat: Enumeració d'estats d'aliança amb un regne.  
 *             Serveix per saber si hi ha aliança activa, pendent o fallida.  
 *  
 ***** */  
  
typedef enum {  
    UNKNOWN,  
    ALLIANCE_NONE,    /* estat per defecte: sense interacció */  
    ALLIANCE_PENDING, /* petició enviada/rebuda i pendent */  
    ALLIANCE_ACTIVE,  /* aliança establerta */  
    ALLIANCE_FAILED   /* petició rebutjada o fallida */  
} AllianceStatus;  
  
/* *****  
 *  
 * @Finalitat: Registre d'aliança associat a un regne concret.  
 *             Guarda el nom del regne, l'estat de l'aliança i el moment  
 *             en què es va iniciar (per gestionar timeouts o històric).  
 *  
 ***** */  
  
typedef struct {  
    char *name;        /* nom del regne (string) */  
    AllianceStatus status; /* estat de l'aliança */  
    time_t requested_at; /* moment de la petició */  
} AllianceRecord;
```

AllianceStatus i AllianceRecord s'utilitzen per representar i gestionar l'estat d'aliança amb cada regne conegut. Aquests elements permeten al sistema saber si una aliança ja està establerta, si hi ha una petició pendent o si una aliança ha fallat o ha estat rebutjada.

Aquesta informació és necessària perquè el Maester pugui recordar l'estat de les relacions amb altres regnes i actuar en conseqüència. Dins del sistema, s'utilitza tant durant la càrrega inicial de rutes com en la gestió de missatges rebuts pel network_controller quan arriben respostes a peticions d'aliança.

Disposar d'aquest registre evita repetir peticions de pledge innecessàries i permet mostrar a l'usuari l'estat actual de les aliances de manera clara.

1.4.6 EnvoyTask i EnvoyResult

Figure 10. Definició de les estructures EnvoyTask i EnvoyResult

```
/* *****
 *
 * @Finalitat: Estructura que representa una tasca enviada del Maester a un Envoy.
 *             Es fa servir per escriure-la directament per pipe (write(fd, &task, sizeof(task))).
 *             Per això utilitza camps de mida fixa (no punters).
 *
 * *****/
typedef struct {
    int type;                // TASK_PLEDGE, TASK_LIST_REQ, TASK_START_TRADE...
    char target_realm[32];   // Nom del regne destí
    char param1[128];        // Paràmetre extra (ex: nom de fitxer), o buit
} EnvoyTask;

/* *****
 *
 * @Finalitat: Missatge de resposta que retorna l'Envoy al Maester.
 *             Inclou si la tasca ha anat bé, un missatge per mostrar i el regne implicat.
 *
 * *****/
typedef struct {
    int success;             // 1 = Èxit, 0 = Fallada
    char message[256];       // Text per imprimir a l'usuari
    char target_realm[32];   // Regne implicat
    int task_type;           // Tipus de tasca completada
} EnvoyResult;
```

Les estructures EnvoyTask i EnvoyResult s'utilitzen com a missatges de control per a la comunicació entre el procés Maester i els processos Envoy mitjançant pipes. A través d'aquestes estructures, el Maester pot enviar tasques als envois i rebre posteriorment el resultat de la seva execució.

Aquestes estructures existeixen perquè l'enviament de dades per pipes resulta més senzill quan es treballa amb paquets de mida fixa, ja que permet utilitzar directament operacions write() i read() sense necessitat de mecanismes de serialització complexos.

Dins del sistema, s'utilitzen principalment al mòdul envoy_controller, en funcions com run_envoy_worker, assign_task_to_envoy i check_envoys_resp.

1.4.7 Envoy

Figure 11. Definició de l'estructura Envoy

```
/* *****
 *
 * @Finalitat: Estructura de gestió d'un Envoy (costat Maester).
 *             Emmagatzema el pid del procés fill i els descriptors de pipe:
 *             - fd_write: per enviar tasques a l'Envoy
 *             - fd_read : per rebre resultats de l'Envoy
 *             També indica si està lliure/ocupat i què està esperant.
 *
 * *****/
typedef struct {
    pid_t pid;                // ID del procés fill

    int fd_write;             // El Maester usa aquest per ENVIAR tasques al fill
    int fd_read;             // El Maester usa aquest per REBRE respostes del fill

    int status;              // ENVOY_FREE o ENVOY_BUSY

    char *waiting_for_realm;  // Regne del qual espera resposta (si aplica)
    int current_task_type;    // Tasca actual assignada
} Envoy;
```

L'estructura Envoy representa un emissor gestionat pel Maester i conté la informació necessària per controlar cada procés Envoy, com el seu PID, els canals d'IPC utilitzats i el seu estat actual.

Aquesta estructura és necessària perquè el Maester pugui saber en tot moment quin envoy es troba lliure i quin està ocupat executant una tasca, així com identificar què està fent cada emissor. Dins del sistema, Envoy s'utilitza per gestionar el pool d'envoys, assignar noves tasques i rebre els resultats un cop aquestes han finalitzat.

Disposar d'aquesta estructura permet implementar un pool d'envoys real, facilitant la gestió de tasques en paral·lel i evitant bloquejos en el procés principal.

1.4.8 GlobalContext

Figure 12. Definició del GlobalContext del programa

```
/* *****
 *
 * @Finalitat: Context global del programa.
 *             Agrupa tota la informació compartida entre mòduls:
 *             regne actual, productes locals, estat del menú, llista de trade,
 *             aliances, recursos de sincronització per prints i gestió d'Envoys.
 *
 * *****/
```

```

typedef struct {
    Kingdom kingdom;

    Product *products;
    int n_products;

    char *main_line;
    char *trade_line;

    TradeList *trade_list;
    int trade_count;

    AllianceRecord *alliances;
    int n_allies;

    semaphore print_sem;
    int print_initialized;

    int pipeServer[2];

    Envoy *envoys;
} GlobalContext;

```

El GlobalContext representa el context global del programa i agrupa tota la informació que és compartida entre els diferents mòduls del sistema. En aquesta estructura s'hi guarden les dades principals necessàries per al funcionament del programa, evitant haver de passar múltiples paràmetres entre funcions.

Aquest context s'utilitza pràcticament a tot el projecte, ja que molts mòduls necessiten accedir a informació comuna. Centralitzar aquestes dades facilita el disseny modular i fa més clara la comunicació entre les diferents parts del sistema.

A més, el GlobalContext permet concentrar en un únic punt tota la memòria que cal alliberar al final de l'execució del programa, simplificant la gestió de recursos.

1.5 Recursos del sistema utilitzats i justificació

En aquest apartat s'expliquen els principals recursos del sistema utilitzats durant el desenvolupament del projecte i la justificació del seu ús dins del disseny global.

1.5.1 Sockets

Els sockets s'utilitzen per gestionar la comunicació entre els diferents regnes del sistema. Com que els regnes poden executar-se en màquines diferents, s'ha optat per utilitzar sockets TCP, que garanteixen una comunicació fiable i ordenada entre els extrems.

Dins del projecte, els sockets s'utilitzen principalment al mòdul `network_controller` per obrir un servidor d'escolta, acceptar connexions entrants i enviar frames a altres regnes. Els usos principals són els següents:

- `open_listen_socket(ip, port)` per obrir el servidor TCP
- `accept()` per rebre connexions entrants
- `send_frame()` per connectar-se a un regne destí i enviar frames
- transferència seqüencial de fitxers durant operacions de trade

1.5.2 Threads

Els threads s'utilitzen per permetre l'execució concurrent del servidor de xarxa sense bloquejar el flux principal del programa. En concret, el servidor s'executa en un thread creat des del procés principal mitjançant la funció `pthread_create`.

Aquest disseny permet que el menú principal continuï atenent les comandes de l'usuari mentre el servidor escolta i processa connexions entrants, fent que el sistema sigui més responsiu.

1.5.3 Forks

La creació de processos mitjançant `fork()` s'utilitza per implementar els envois, que representen emissors independents encarregats d'executar tasques que poden ser llargues o bloquejants.

Aquests processos permeten executar diverses operacions en paral·lel, com l'enviament de peticions de pledge o la gestió de trades, mantenint una separació clara de tasques i evitant bloquejos en el procés principal.

1.5.4 Pipes

Les pipes s'utilitzen com a mecanisme d'IPC per a la comunicació entre el Maester i els processos Envoy. Per a cada envoy es defineixen dos canals de comunicació: un per enviar tasques del Maester a l'envoy i un altre per rebre'n els resultats.

Aquest mecanisme, basat en `read()` i `write()`, és suficient per transmetre estructures de dades petites com `EnvoyTask` i `EnvoyResult`, així com senyals interns, com ara la notificació del Ctrl+C mitjançant una pipe específica.

1.5.5 Select

La crida `select()` s'utilitza principalment dins dels processos Envoy per esperar dades amb un temps límit definit. Aquest mecanisme permet implementar timeouts de manera controlada, evitant l'ús de crides bloquejants com `sleep()`.

Gràcies a `select()`, els envois poden esperar respostes de xarxa sense quedar bloquejats indefinidament.

1.5.6 Senyals

Els senyals s'utilitzen per gestionar la finalització controlada del programa. En concret, el procés principal captura el senyal SIGINT (Ctrl+C) per permetre una sortida neta del sistema.

Aquesta gestió evita deixar recursos oberts, com sockets, pipes o semàfors, i permet alliberar correctament la memòria abans de finalitzar el programa.

1.5.7 Semàfors

Els semàfors s'utilitzen per garantir una sortida per pantalla coherent quan diversos threads o processos escriuen simultàniament. En el projecte, el mòdul `safe_print` fa servir un semàfor binari definit a `semaphore_v2.h`.

Abans d'imprimir per pantalla, qualsevol fil o procés fa un `SEM_wait`, i un cop finalitzada la impressió fa un `SEM_signal`. Aquest mecanisme evita que els missatges apareguin barrejats i millora la llegibilitat de la sortida.

1.5.8 System calls i utilitats

A més, el projecte fa ús de diverses system calls i funcions auxiliars per gestionar la lectura i escriptura de dades. La funció `read_until()` s'utilitza per llegir des d'un file descriptor fins a trobar un delimitador concret, mentre que les crides `open()`, `read()`, `write()` i `close()` es fan servir tant per a la gestió de fitxers com per a la comunicació entre processos.

Per al càlcul del hash dels fitxers, s'utilitza l'eina `md5sum`, executada mitjançant un procés fill creat amb `fork()` i `execlp()`, comunicant-se amb el procés pare a través d'una pipe. Aquest enfocament permet mantenir un estil basat en system calls i evitar dependències externes addicionals.

1.6 Fluxos principals

El sistema es controla principalment a través d'un menú principal, gestionat pel mòdul `main_controller`, que permet a l'usuari interactuar amb el regne i executar les operacions disponibles. A través d'aquest menú es poden consultar regnes, gestionar aliances i iniciar operacions de comerç. Les principals comandes disponibles són les següents:

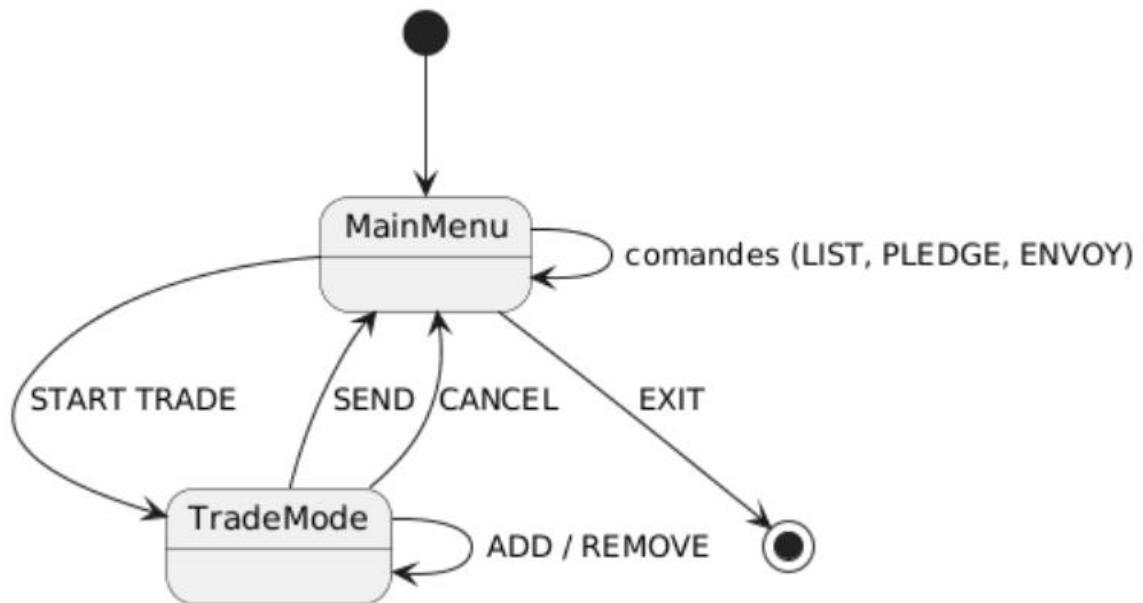
- `LIST REALMS`: mostra la llista de regnes coneguts a partir de les rutes carregades
- `LIST PRODUCTS [realm?]`: permet llistar els productes locals o els d'un regne remot mitjançant una petició gestionada per un envoy
- `PLEDGE <realm> <sigil>`: envia una petició d'aliança a un altre regne
- `START TRADE <realm>`: inicia el mode trade amb un regne aliat
- `ENVOY STATUS`: mostra l'estat actual del pool d'envoys
- `EXIT`: finalitza l'execució del programa

Quan l'usuari inicia una operació de trade, el sistema entra en un mode específic gestionat pel mòdul `trade_controller`. En aquest mode, l'usuari pot construir una llista temporal de productes abans d'enviar-la al regne destí. Les comandes disponibles en aquest mode són les següents:

- `ADD <producte> <quantitat>`: afegeix un producte a la llista de trade, validant-lo amb el catàleg remot guardat en un fitxer temporal

- REMOVE <id>: elimina un element de la llista de trade a partir del seu índex
- SEND: genera el fitxer temporal de trade i assigna la tasca a un envoy
- CANCEL: buida la llista de trade i retorna al menú principal

Figure 13. Diagrama d'estats del menú principal i del mode trade.



2. Problemes observats i com s'han solucionat

Durant el desenvolupament de la pràctica s'han detectat diversos problemes, tant a nivell de disseny com d'implementació. A continuació es descriuen els principals problemes trobats i la solució adoptada en cada cas.

2.1 Bloqueig del programa per operacions de xarxa

En les primeres versions del programa, les operacions de xarxa s'executaven dins del mateix flux que el menú principal. Aquest plantejament provocava que el programa quedés bloquejat mentre esperava connexions entrants o respostes de xarxa, impedit que l'usuari pogués continuar introduint comandes per terminal.

Per solucionar aquest problema, es va decidir separar la lògica de xarxa del menú principal mitjançant l'ús d'un thread servidor. Aquest thread s'encarrega exclusivament d'obrir el socket d'escolta, acceptar les connexions entrants i processar els frames rebuts, de manera independent al flux principal.

Gràcies a aquesta separació, el menú principal pot continuar funcionant de forma interactiva mentre el servidor gestiona la comunicació en paral·lel, evitant bloquejos i millorant l'experiència d'ús del sistema.

2.2 Problemes de concurrència en la sortida per pantalla

Quan el sistema va començar a utilitzar diversos fils d'execució, es va detectar que la sortida per pantalla apareixia desordenada, amb textos barrejats o incomplets. Aquest problema es produïa perquè diferents fils escrivien simultàniament a la sortida estàndard sense cap mecanisme de sincronització.

Per solucionar-ho, es va implementar un sistema d'impressió segura mitjançant un semàfor binari, encapsulat al mòdul `safe_print`. Abans d'escriure per pantalla, qualsevol fil fa un `SEM_wait` per obtenir accés exclusiu, imprimeix el missatge corresponent i, un cop finalitzat, fa un `SEM_signal`.

Aquesta solució garanteix que només un fil escriu a la sortida estàndard al mateix temps, mantenint una sortida clara i coherent.

2.3 Gestió complexa de comunicació amb els envois

La comunicació entre el Maester i els processos Envoy va resultar inicialment complexa a causa d'un ús incorrecte de pipes bidireccionals. Aquest enfocament provocava lectures incorrectes i, en alguns casos, bloquejos durant la comunicació entre processos.

Per resoldre aquest problema, es va optar per separar clarament els canals de comunicació, utilitzant un descriptor exclusiu per enviar tasques del Maester a l'Envoy i un altre descriptor diferent per rebre les respostes generades per l'Envoy. A més, les estructures `EnvoyTask` i `EnvoyResult` es van dissenyar amb camps de mida fixa, fet que permet utilitzar directament les crides `write()` i `read()` sense necessitat de processos de serialització complexos.

Aquest canvi ha simplificat la comunicació entre processos i ha fet el sistema més robust.

2.4 Validació del protocol i corrupció de dades

Durant les proves de comunicació entre regnes, es van detectar situacions en què els frames rebuts contenien dades incorrectes o incompletes, especialment en entorns de xarxa no fiables.

Per evitar processar missatges corruptes, es va implementar un mecanisme de validació mitjançant un checksum dins del protocol de comunicació. Durant la serialització del frame es calcula la suma dels bytes, i durant la deserialització es valida aquest checksum abans de processar el contingut del missatge.

Gràcies a aquesta validació, els frames incorrectes es descarten automàticament, millorant la fiabilitat de la comunicació.

2.5 Gestió de memòria dinàmica i fuites

Durant el desenvolupament del projecte es van detectar fuites de memòria, sobretot en l'ús de funcions com `malloc`, `realloc` i `asprintf`, així com en estructures dinàmiques com les llistes de trade o les rutes.

Per solucionar aquest problema, es va fer una revisió exhaustiva del codi per assegurar que cada reserva de memòria tingués el seu corresponent free. També es va garantir que les estructures dinàmiques es reinicialitzessin correctament quan deixaven de ser necessàries.

Finalment, en sortir del programa, es crida a una funció centralitzada encarregada d'alliberar tots els recursos utilitzats, evitant fuites de memòria.

2.6 Control del flux del mode trade

Durant la implementació del mode trade es van detectar situacions en què el sistema podia quedar en un estat inconsistent, especialment quan l'usuari cancel·lava un trade o intentava enviar una llista buida.

Per evitar aquests casos, es van afegir comprovacions explícites per assegurar que no es pugui executar l'ordre SEND si la llista de trade està buida. A més, es garanteix que la llista de trade s'allibera correctament tant en els casos de SEND com de CANCEL, i que el sistema retorna sempre al menú principal de manera controlada.

Aquestes comprovacions asseguruen un flux coherent del mode trade.

2.7 Gestió de senyals

Inicialment, quan l'usuari prems Ctrl+C, el programa finalitzava de manera abrupta, deixant recursos oberts com pipes, sockets o semàfors.

Per solucionar-ho, es va implementar un handler del senyal SIGINT que permet gestionar la finalització del programa de manera controlada. Aquest handler notifica el fil principal mitjançant una pipe, permet sortir del bucle principal i alliberar tots els recursos abans de finalitzar l'execució.

Aquesta gestió evita deixar recursos oberts i garanteix una sortida neta del sistema.

2.8 Dificultat en el disseny global del sistema

El projecte combina múltiples conceptes de sistemes operatius, com ara processos, threads, sockets, pipes, semàfors i un protocol de comunicació propi, fet que va complicar el disseny inicial i la integració de totes les parts.

Per afrontar aquesta dificultat, es va optar per un disseny modular clar, amb una separació per controladors i una definició clara de les responsabilitats de cada mòdul.

A més, es va utilitzar un GlobalContext per centralitzar l'estat del programa i facilitar la comunicació entre components.

Aquest enfocament ha facilitat tant el manteniment del codi com la comprensió del funcionament global del sistema.

3. Estimació temporal del desenvolupament

La pràctica s'ha desenvolupat de manera col·laborativa entre els dos membres del grup.

Tots dos hem participat tant en la fase de disseny com en la implementació i el testing, tot i que en alguns moments un membre s'ha pogut centrar més en una part concreta.

<i>Categoria</i>	<i>Alex Ortiz</i>	<i>Bruno Xampeny</i>
<i>Investigació</i>	6 h	8 h
<i>Disseny</i>	3 h	4 h
<i>Implementació</i>	14 h	20 h
<i>Testing</i>	4 h	3 h
<i>Documentació</i>	4 h	2 h
<i>Total aprox.</i>	30 h	37 h

4. Conclusions i propostes de millora

La realització d'aquesta pràctica ha permès aplicar de manera pràctica molts dels conceptes treballats a l'assignatura, especialment la gestió de processos, la comunicació entre processos, l'ús de sockets TCP i la sincronització mitjançant semàfors.

Un dels aspectes més positius del projecte ha estat el disseny modular del programa. La separació del codi en diferents controladors ha facilitat la comprensió del sistema i ha permès treballar de manera més ordenada i estructurada. A més, l'ús d'un protocol de comunicació basat en frames de mida fixa ha simplificat la lectura i escriptura per sockets i ha reduït possibles errors durant la deserialització.

També ha estat especialment interessant la implementació del sistema d'envoys, ja que combina processos fills, pipes i coordinació amb el procés principal. Aquest enfocament ha ajudat a entendre millor com gestionar tasques asíncrones sense bloquejar el menú principal.

Pel que fa a les dificultats trobades, un dels punts més complexos ha estat la sincronització entre el thread servidor, el procés principal i els envoys. En alguns moments es van produir bloquejos o comportaments inesperats, que es van solucionar mitjançant una separació més clara de responsabilitats i la incorporació de mecanismes de sincronització com semàfors i canals de comunicació diferenciats.

Com a possibles millores futures del projecte, es podrien considerar les següents:

- Implementar un sistema de logs per guardar l'historial de comunicacions i errors.
- Millorar la gestió d'errors de xarxa, especialment en connexions fallides o talls sobtats.
- Ampliar el sistema de trade amb confirmacions múltiples o validacions addicionals.
- Afegir una interfície més amigable.

Com a conclusió final, considerem que la pràctica ha estat molt completa i útil per consolidar coneixements tant teòrics com pràctics.

5. Explicació dels noms dels processos i temàtica de la pràctica

La pràctica d'aquest curs 2025-26 s'ha basat en l'univers de la saga House of the Dragon, dins del context polític i social derivat de Game of Thrones. Aquest univers es caracteritza per la fragmentació del poder, les aliances inestables entre cases nobles i la importància de la comunicació i la diplomàcia per garantir la supervivència dels regnes. Aquesta temàtica encaixa de manera natural amb un sistema distribuït format per diferents entitats que han de comunicar-se i cooperar entre elles.

Procés Maester

El Maester representa el procés principal de cada regne. És el responsable de gestionar la interacció amb l'usuari, coordinar les aliances amb altres regnes, iniciar processos de comerç, assignar tasques als processos auxiliars i mantenir l'estat global del regne.

De la mateixa manera que un Maester assessora una casa noble i actua com a intermediari, aquest procés centralitza la lògica del sistema i garanteix que totes les operacions es facin de manera ordenada i controlada.

Processos Envoy

Els Envoys representen els enviats o missatgers del regne. En l'univers de la pràctica, són els encarregats de viatjar entre castells per portar informació, peticions o respostes.

A nivell tècnic, cada Envoy és un procés fill creat pel Maester mitjançant `fork()`, que s'encarrega de gestionar la comunicació amb altres regnes de manera independent, enviant i rebent frames per socket i executant tasques concretes com la petició

d'aliances, la sol·licitud de catàlegs de productes o la gestió de negocis de comerç.

Aquesta separació permet que el Maester no quedi bloquejat mentre s'estan realitzant comunicacions de xarxa, fent el sistema més robust i escalable.

Altres conceptes del sistema

Els Realms representen les diferents Grans Cases, cadascuna executant el seu propi Maester. Les aliances simbolitzen els pactes polítics entre cases, i només amb una aliança activa es permet iniciar operacions de comerç.

Les rutes reflecteixen la geografia del continent, ja que els missatges poden haver de passar per diversos regnes intermedis abans d'arribar al destí. Finalment, el comerç representa els acords comercials entre regnes aliats, gestionats mitjançant llistes de productes i enviats a través d'Envoys.

En conclusió, The Citadel System utilitza una narrativa inspirada en *House of the Dragon* per donar sentit a un sistema distribuït i descentralitzat facilitant la comprensió del disseny i reforçant l'aprenentatge dels mecanismes fonamentals dels sistemes operatius.

6. Bibliografia utilitzada

Apunts i material docent

La Salle Universitat Ramon Llull. (s.d.). *Introducció al llenguatge C*. Barcelona: La Salle.

La Salle Universitat Ramon Llull. (s.d.). *Tema 5.1. File Descriptors i Signals*. Barcelona: La Salle.

La Salle Universitat Ramon Llull. (s.d.). *Tema 5.2. Forks*. Barcelona: La Salle.

La Salle Universitat Ramon Llull. (s.d.). *Tema 5.3. Threads*. Barcelona: La Salle.

La Salle Universitat Ramon Llull. (s.d.). *Tema 5.4. Sockets*. Barcelona: La Salle.

La Salle Universitat Ramon Llull. (s.d.). *Tema 5.5. Pipes*. Barcelona: La Salle.

La Salle Universitat Ramon Llull. (s.d.). *Tema 5.6. Memòria Compartida*. Barcelona: La Salle.

La Salle Universitat Ramon Llull. (s.d.). *Tema 5.7. Cues de Missatges*. Barcelona: La Salle.

La Salle Universitat Ramon Llull. (s.d.). *Tema 5.8. Select*. Barcelona: La Salle.

La Salle Universitat Ramon Llull. (s.d.). *Assignació de Ports*. Barcelona: La Salle.

La Salle Universitat Ramon Llull. (s.d.). *Guia d'estils de programació en C*. Barcelona: La Salle.

Exemples de codi utilitzats a classe

La Salle Universitat Ramon Llull. (s.d.). *Exemples de signals* (signals_1.c, signals_2.c, signals_3.c).

La Salle Universitat Ramon Llull. (s.d.). *Exemples de fork* (forkejant.c, forkejant1.c, forkejant2.c, forkejant3.c, forkejant4.c, forkejant5.c).

La Salle Universitat Ramon Llull. (s.d.). *Exemples de pipes* (exemple1.c, exemple2.c, exemple3.c, exemple4.c).

La Salle Universitat Ramon Llull. (s.d.). *Exemples de sockets client-servidor* (client.c, server.c).

La Salle Universitat Ramon Llull. (s.d.). *Exemples de select* (example1.c, example2.c, example3.c).